

Cuestionario teórico

Sebastian Canteros - Facundo Nestasio

1) Sistemas Operativos- Generalidades

a) ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?

Es el scheduler, su función principal es la de organizar las tareas y otorgarles un tiempo de ejecución.

b) Explique a qué se denomina “Contexto de Ejecución”. ¿Cuáles son las variables intervinientes?

El contexto de ejecución es el estado del procesador y la memoria en un instante preciso. Las variables intermitentes son los registros, el contador y la pila. Son vitales para saber dónde están las variables locales y las direcciones de retorno de funciones.

c) ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

Se denomina Sistema Operativo de Tiempo Real a aquel que debe responder en un tiempo límite establecido y para ello utiliza múltiples hilos de procesamiento. La ventaja de usar uno es que puede procesar varias tareas en paralelo gracias a sus múltiples hilos.

2) Kernel de FreeRTOS

a) Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

Una tarea es un pequeño programa. Tiene un punto de entrada y corre en un lazo infinito. No deben retornar a su función de implementación. Si no es necesaria podrá ser borrada.

```
void vSumaTask(void *pvParameters)
{
    int a = 5;
    int b = 10;
    int c;
    for ( ; ; )
    {
        c = a + b;
    }

    vTaskDelete(NULL);
}
```

b) Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.

Las tareas tienen 3 estados posibles, Ready, Blocked, Suspended y Running. El estado Ready indica que una tarea está lista para ejecutarse, el estado Running indica que la tarea se está ejecutando, la tarea Blocked indica que una tarea está en el estado de espera y por último la tarea suspended indica que la tarea está totalmente inactiva. Las funciones que permiten estas transiciones son `vtasksuspend`, `vtaskresume`, `event` y `vtask delay`.

c) ¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”? ¿y a qué nos referimos con el concepto de “tareas manejadas por eventos”?

Las tareas de procesamiento continuo son tareas que están constantemente consumiendo recursos del CPU, debido a que están la mayor parte del tiempo en estado ready. Las tareas manejadas por eventos son aquellas tareas que pasan a estado ready cuando se recibe un dato o una interrupción.

d) ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

Una tarea puede estar en estado bloqueado por estar esperando datos, cola llena, cola vacía. También por esperar eventos temporales como un `vtaskdelay` y por último por un evento de sincronización como un semáforo.

e) ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

La principal función de la Idle task es mantener al procesador en un estado mínimo para evitar que en el caso de que todas las tareas estén bloqueadas al mismo tiempo, el procesador no pueda poner en marcha una tarea en ready.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

Cuando se envía un dato a través de una cola se hace una copia del dato original y lo guarda de forma local en la cola. Al crear una cola se debe aclarar el tipo de dato que albergará y el tamaño de cada uno de esos datos. Además las colas albergan datos con la estructura FIFO donde el primer dato en entrar es el primer dato en salir. La escritura en estas colas se realiza con `queue send` y `queue receive`.

4) Explique utilizando un ejemplo con código el porqué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

Se usan múltiples tareas escritoras y una sola tarea lectora debido a que esto facilita la protección de los recursos compartidos debido a que, los datos se envían y se leen de manera ordenada. Si tuviéramos más de una tarea lectora para un mismo recurso esto haría que se empezaran a mezclar los datos y no se transmitan de forma adecuada.

```
#include "sdkconfig.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
```

```
typedef struct {
    uint8_t id_sensor;
    float medicion;
} Mensaje_t;
```

```
QueueHandle_t xColaDatos;
```

```
void vTareaSensor(void *pvParameters) {
```

```

Mensaje_t msj;
msj.id_sensor = (uint8_t)(uint32_t)pvParameters;

for(;;) {
    msj.medicion = LeerSensor(msj.id_sensor);

    xQueueSend(xColaDatos, &msj, portMAX_DELAY);

    vTaskDelay(pdMS_TO_TICKS(500));
}
}

void vTareaPantalla(void *pvParameters) {
    Mensaje_t msjRecibido;

    for(;;) {

        if( xQueueReceive(xColaDatos, &msjRecibido, portMAX_DELAY) == pdTRUE ) {

            printf("Sensor %d midio: %.2f\n", msjRecibido.id_sensor, msjRecibido.medicion);
        }
    }
}

```

Cuando se ejecuta queue receive y no hay datos en la cola la tarea pasa del estado running al estado blocked. Además si se ejecuta un queuesend y la cola esta llena, la tarea quedará bloqueada hasta que haya espacio en la cola.

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM?

Para enviar datos compuestos el proceso que se lleva a cabo es enviar una estructura que contenga los datos que se quieren enviar. Esto no generaría conflictos dado que las colas deben estar compuestas siempre por datos del mismo tipo y al ser estructuras esto se cumple.

En el caso de que los datos fuesen grandes, para no tener problemas con la memoria RAM lo recomendable sería enviar punteros hacia los datos a las colas en lugar de los datos en sí, lo que haría más eficiente el tiempo de procesamiento.

6) Explicar el funcionamiento del esquema de la fig 1:

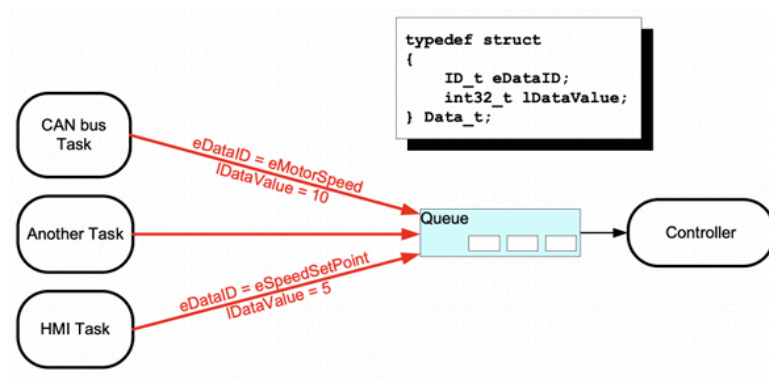


Fig. 1. Recibir datos de múltiples fuentes

En la figura 1 podemos ver una cola de múltiples entradas que recibe datos del tipo struct de varios emisores y los envía a una tarea controladora. La estructura contiene en sus campos, además de los datos, la fuente de estos para poder determinar como deben ser procesados.

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"

```

```
SemaphoreHandle_t xMutexSerial;
```

```

void vTareaImpresora(void *pvParameters)
{
    char *pcTareaNombre = (char *)pvParameters;

    for (;;)
    {
        if (xSemaphoreTake(xMutexSerial, pdMS_TO_TICKS(1000)) == pdTRUE)
        {
            printf("La tarea %s está usando el recurso Serial.\n", pcTareaNombre);
            vTaskDelay(pdMS_TO_TICKS(500));
            printf("La tarea %s ha liberado el recurso.\n", pcTareaNombre);

            xSemaphoreGive(xMutexSerial);
        }
        else
        {
            printf("La tarea %s no pudo obtener el Mutex, intentará luego.\n", pcTareaNombre);
        }

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void app_main()
{

```

```

xMutexSerial = xSemaphoreCreateMutex();

if (xMutexSerial != NULL)
{
    xTaskCreate(vTareaImpresora, "Tarea1", 2048, "A", 1, NULL);
    xTaskCreate(vTareaImpresora, "Tarea2", 2048, "B", 1, NULL);
}
}

```

Dos tareas intentan escribir en el puerto serie. Sin el mutex, que se apropia de ese recurso compartido, los mensajes se mezclarían.

8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad” como se ve en la fig 2. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador. Fig. 2. Inversión de prioridad permitan comprenderlo.

```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"

```

```

SemaphoreHandle_t xMutexRecurso;

```

```

void vTareaBaja(void *pvParameters) {
    for(;;) {
        xSemaphoreTake(xMutexRecurso, portMAX_DELAY); // Toma el recurso
        printf("Baja: Tengo el recurso. Trabajando...\n");
        vTaskDelay(pdMS_TO_TICKS(500)); // Simulamos trabajo largo
        xSemaphoreGive(xMutexRecurso); // Suelta el recurso
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

```

void vTareaMedia(void *pvParameters) {
    vTaskDelay(pdMS_TO_TICKS(100)); // Espera a que la Baja tome el mutex
    for(;;) {
        printf("Media: Ejecutando tarea intensa sin recurso...\n");
        // Esta tarea no necesita el mutex, pero acapara la CPU
        vTaskDelay(pdMS_TO_TICKS(200));
    }
}

```

```

void vTareaAlta(void *pvParameters) {
    vTaskDelay(pdMS_TO_TICKS(200)); // Espera a que Baja tome el mutex
    for(;;) {
        printf("Alta: Intentando tomar recurso...\n");
        xSemaphoreTake(xMutexRecurso, portMAX_DELAY); // SE BLOQUEA AQUÍ
        printf("Alta: Recurso obtenido.\n");
        xSemaphoreGive(xMutexRecurso);
    }
}

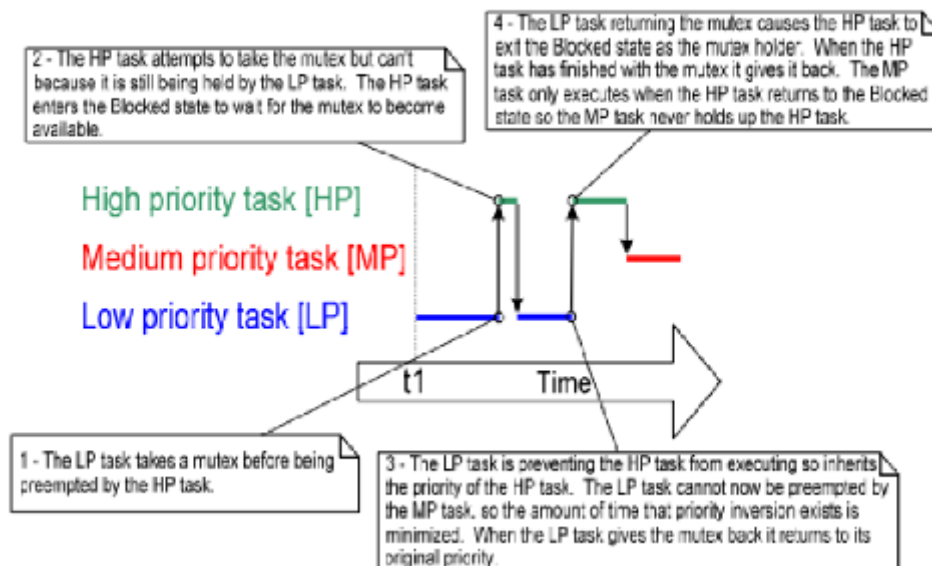
```

```

    vTaskDelay(pdMS_TO_TICKS(100));
}
}

```

9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.



La herencia compartida es un mecanismo utilizado para resolver la inversión de prioridad, donde una tarea de alta prioridad queda bloqueada indefinidamente porque una tarea de baja prioridad retiene un recurso compartido.

- 1 - La tarea de baja prioridad toma el mutex y luego es adelantada por una tarea de alta prioridad.
- 2 - La tarea de alta prioridad intenta tomar el mutex pero no puede, porque lo tiene la tarea de baja prioridad, se bloquea y espera a que el mutex esté disponible.
- 3 - La tarea de baja prioridad está impidiendo que la tarea de alta prioridad se ejecute, entonces, le da la misma prioridad que la tarea de alta prioridad para evitar que la tarea de prioridad media pueda adelantarse. Con esto se logra minimizar el tiempo en el que existe la inversión de prioridad. Cuando la tarea de baja prioridad devuelve el mutex, esta recupera su prioridad original.
- 4 - Cuando la tarea de baja prioridad devuelve el mutex, este es tomado por la tarea de alta prioridad y abandona el estado bloqueado. Cuando la tarea de alta prioridad termina de usar el mutex lo devuelve. La tarea de prioridad media solo se ejecutará cuando la tarea de alta prioridad pase a estar bloqueada nuevamente.

10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

```

QueueHandle_t cola;
SemaphoreHandle_t uartMutex;

void TareaA(void *pvParameters)
{

```

```

int dato;

while(1)
{
/* Toma el recurso */
xSemaphoreTake(uartMutex, portMAX_DELAY);

printf("A: Tengo el UART\n");

/* Espera un dato de la cola */
xQueueReceive(cola, &dato, portMAX_DELAY);

printf("A: Recibi %d\n", dato);

xSemaphoreGive(uartMutex);
}

void TareaB(void *pvParameters)
{
int dato = 100;

while(1)
{
/* También necesita el UART */
xSemaphoreTake(uartMutex, portMAX_DELAY);

printf("B: Voy a enviar dato\n");

xQueueSend(cola, &dato, portMAX_DELAY);

xSemaphoreGive(uartMutex);

vTaskDelay(pdMS_TO_TICKS(1000));
}
}

int main(void)
{
uartMutex = xSemaphoreCreateMutex();
cola = xQueueCreate(5, sizeof(int));

xTaskCreate(TareaA, "A", 256, NULL, 2, NULL);
xTaskCreate(TareaB, "B", 256, NULL, 2, NULL);

vTaskStartScheduler();

while(1);
}

```